

CoBRAC User Guide

Version: CoBRAC-v1-0

1. Introduction

Related files

CoBRAC files (instruction documents, conversion scripts, etc.) are available on GitHub:
<https://github.com/miyamoto9265/CoBRAC>.

What is CoBRAC?

CoBRAC (Collaborative Brain Reference Architecture Compiler) is a workflow for structurally describing information processing in a brain region (ROI) that implements a computational function (TLF), based on neuroscientific evidence. The final deliverable is **BRA** (**B**rain **R**eference **A**rchitecture) data, exported as an Excel file with the extension `.bra.xlsx`.

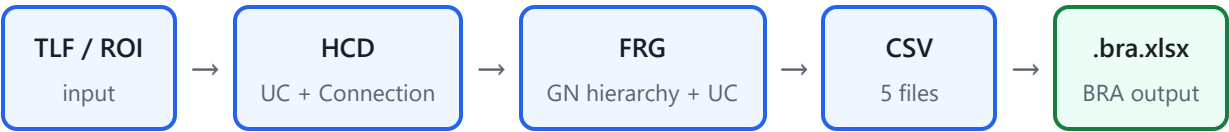


Figure 1-1 — CoBRAC overall flow

Final deliverable (.bra.xlsx)

The Excel file contains five sheets:

Sheet	Content
Project	Project metadata (Contributor, Project ID, etc.)
References	Reference list
Circuits	Uniform Circuit (UC) definitions
Connections	Connections between UCs
FRG	Function Realization Graph integrated with UC data

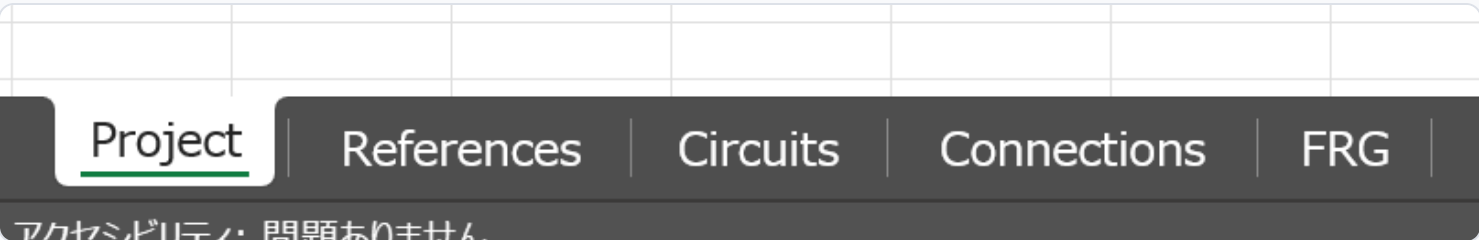


Figure 1-2 — Completed .bra.xlsx sheet structure

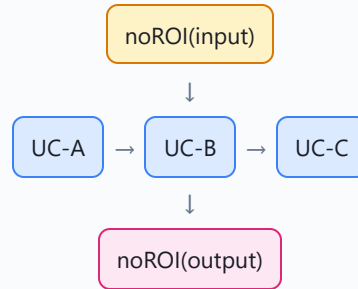
How to use this guide

1. First-time users: read Chapters 2–4 for concepts and workflow
2. Start work: follow Chapters 5 onward in Phase order
3. Detailed writing rules: have your AI agent read the relevant `instruction_*.md` files
4. Before submission: use the checklist in Chapter 11

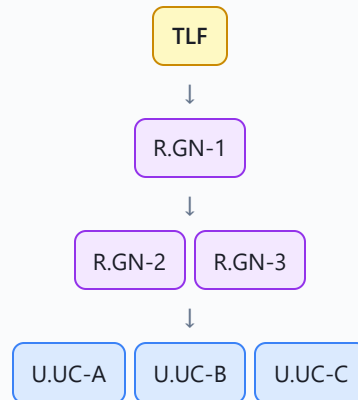
2. Core concepts

Term	Description
TLF (Top Level Function)	The computational function to describe (e.g., VOR learning, spatial navigation)
ROI (Region of Interest)	Brain region where the TLF is implemented (e.g., cerebellum, hippocampus)
UC (Uniform Circuit)	Basic unit of HCD (neural population). “Circuit” is a project-specific term, not a synaptic circuit
BIF (Brain Information Flow)	Anatomical projection data supporting connections
Connection	Information transfer between UCs based on BIF
Interface	UC input/output relation: <code>([outputs], ...) = UC-name([inputs], ...)</code>
Output Semantics	Meaning of information coded by a UC
GN (Group Node)	Intermediate FRG node representing a sub-function of the TLF
HCD	Hypothetical Component Graph — information-processing graph of UCs and Connections
FRG	Function Realization Graph — hierarchical decomposition of TLF grounded in UCs

HCD — “Who connects to whom”



FRG — “How function decomposes into UCs”



 UC inside ROI
 noROI(input)
 noROI(output)
 GN

Figure 2-1 — HCD vs FRG (the same UCs appear in both)

Interface notation example

$([UC-E], [UC-H]) = UC-B([UC-F], [UC-G], [UC-I])$

Figure 2-2 — UC-B takes UC-F, UC-G, UC-I as inputs and outputs to UC-E and UC-H

3. Prerequisites

Required software

- **Python 3.x** — for CSV → xlsx conversion
- **pandas, openpyxl** — `pip install -r requirements.txt`
- **AI agent** — Cursor, Claude Code, Codex, etc. (see Chapter 10)
- **Web browser** — for literature search
- **(Optional) draw.io** — for HCD diagram visualization

Repository layout (`_latest_version`)

```
_latest_version/  
├─ instruction_0.md      # Folder setup  
├─ instruction_1_HCD.md  # HCD (8 steps)  
├─ instruction_2_FRG.md  # FRG (6 steps)  
├─ instruction_3_csv.md  # CSV creation  
├─ csv_to_excel.py       # xlsx generation  
└─ {ProjectName}/       # Per-project folders
```

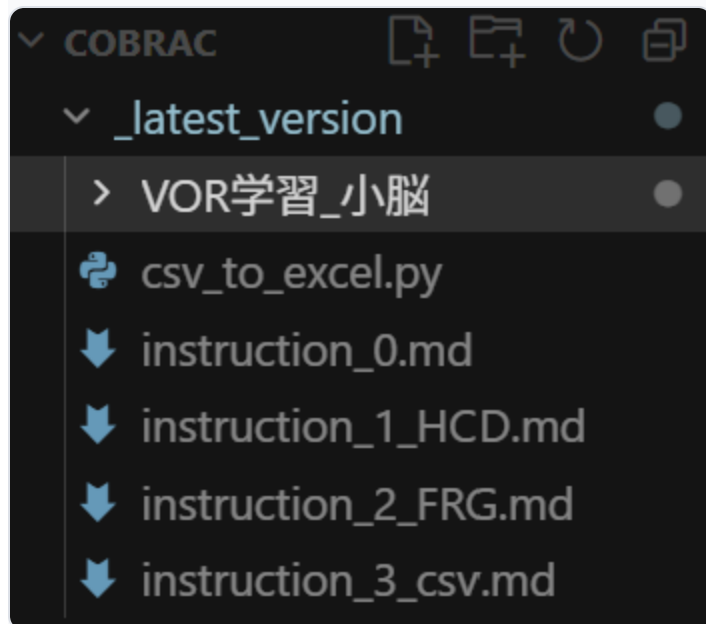


Figure 3-1 — `_latest_version` folder layout

Information to decide before starting

- **Contributor** — author name (English)
- **Project ID** — project name (English, reflecting TLF/ROI)

- **TLF** and **ROI** — target function and brain region

4. Overall workflow

1. `instruction_0.md` — create project folders
2. `instruction_1_HCD.md` — create HCD
3. `instruction_2_FRG.md` — create FRG
4. `instruction_3_csv.md` — create CSV files
5. `csv_to_excel.py` — generate `.bra.xlsx`

```
{ProjectName}/  
├─ {ProjectName}_HCD/    ← HCD outputs (8 md files)  
├─ {ProjectName}_FRG/    ← FRG outputs (5 md files)  
└─ {ProjectName}_CSV/    ← 5 CSV + output xlsx
```


Phase 0

Start

- instruction_0
- folders

Phase 1

HCD

- 1_Thinking ... 8_diagram
- 3_UC, 4_Connection

Phase 2

FRG

- 1_初期分解 ... report
- 3_最終FRG

Phase 3

CSV

- 5 CSV files

Phase 4

xlsx

- csv_to_excel.py
- .bra.xlsx

Figure 4-1 — Phase timeline

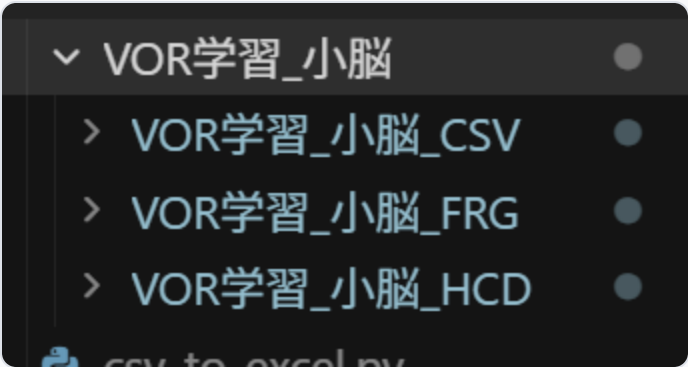


Figure 4-2 — Example completed project folder

Phase	Input	Output
0	TLF, ROI	3 subfolders

Phase	Input	Output
1 HCD	TLF, ROI, literature	8 md files
2 FRG	TLF, UCs from HCD	5 md files
3 CSV	HCD + FRG md	5 CSV files
4 xlsx	5 CSV files	.bra.xlsx

5. Phase 0 — Project start

Project ID naming

- English ID reflecting TLF/ROI (e.g. Cerebellum-VOR-Learning)
- No spaces; Kebab-case recommended

Example prompt to an AI agent

Following instruction_0.md, create project folders with
Project ID: Cerebellum-VOR-Learning,
ROI: cerebellum (flocculus), TLF: VOR learning.

↓ instruction_0.md

の作業を開始してください。

ROI: 小脳

TLF: VOR学習|

∞ Agent ▾

Composer 2.5 Fast ▾

↑

Figure 5-1 — Example Phase 0 AI prompt

6. Phase 1 — Creating HCD

For detailed writing rules, refer to `instruction_1_HCD.md` . This chapter summarizes each step only.

Step	Output	Summary
1	<code>1_Thinking.md</code>	Validate ROI/TLF; organize ROI inputs/outputs
2	<code>2_BIF.md</code>	Literature survey of neural connections; reference list
3	<code>3_UC.md</code>	Define UCs (Kebab-case; inside/outside ROI)
4	<code>4_Connection.md</code> + <code>3_UC.md</code>	Define Connections; add Interfaces
5	<code>3_UC.md</code>	Output Semantics
6	<code>3_UC.md</code>	Requirement / Capability / Mechanism / Implementation — see instruction
7	<code>5_Verification.md</code>	Consistency verification
8	<code>6_FinalPaper.md</code> etc.	Report, tutorial, Mermaid diagram

About Step 6

Writing Requirement, Capability, Mechanism, and Implementation is complex. Always have your AI agent read Step 6 in `instruction_1_HCD.md` and follow it.

Key points

- ☐ Wrap UC names in backticks (` `) in markdown
- ☐ Mark outside-ROI UCs with `noROI(input)` / `noROI(output)` in comments
- ☐ Connections must match Interfaces
- ☐ Every Connection must cite literature

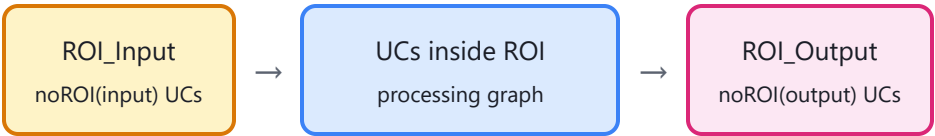


Figure 6-2 — Information flow in HCD (ROI input → internal processing → ROI output)

HCD structure (as in 8_diagram.md)

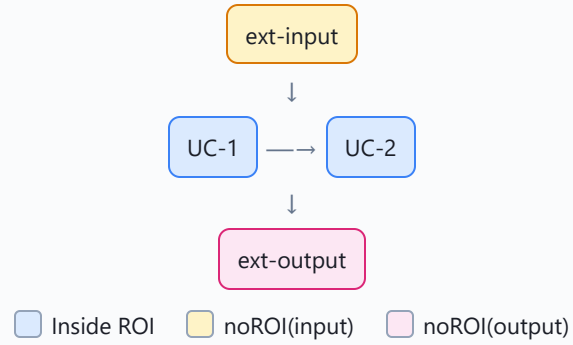


Figure 6-3 — HCD color legend (actual diagram from 8_diagram.md Mermaid via draw.io, etc.)

7. Phase 2 — Creating FRG

See `instruction_2_FRG.md` . Use only UCs inside the ROI as defined in HCD.

Step	Output	Summary
1	<code>1_初期分解結果.md</code>	Decompose TLF (no UC linking yet)
2	<code>2_最適化FRG.md</code>	Merge redundant GNs
3	<code>3_最終FRG.md</code>	Link UCs; add Interfaces
4	<code>4_機能詳細.md</code>	TLF/GN Requirement etc. — see instruction
5	<code>5_FRG作成レポート.md</code>	Document rationale

GN–UC connection constraints (required)

- Each GN must decompose into **multiple UCs**
- At most **2 UCs** per GN
- At most **2 GNs** per UC
- If violated, return to Step 1 and refine TLF decomposition

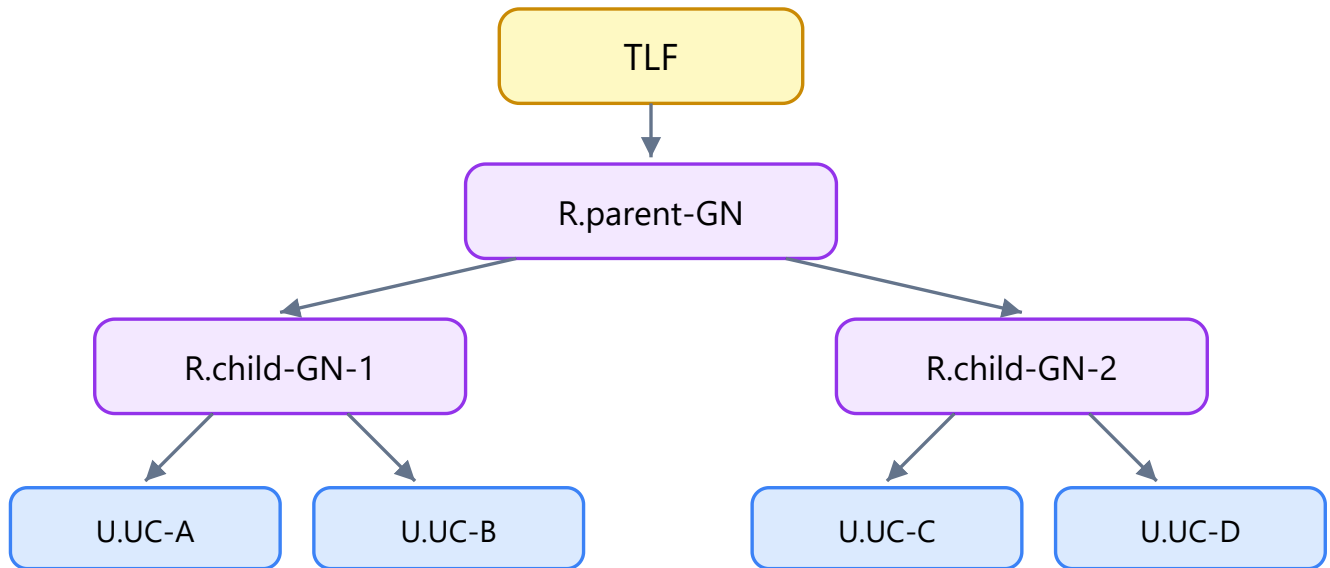
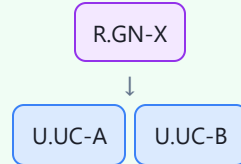
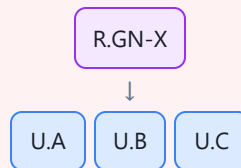


Figure 7-1 — FRG hierarchy (TLF → GN → UC)

OK — 1 GN → at most 2 UCs



NG — 1 GN → 3 UCs



→ Return to Step 1 and refine decomposition

Figure 7-2 — GN-UC connection constraints

GN naming: **R.** + Kebab-case. UC reference: **U.** + Circuit ID.

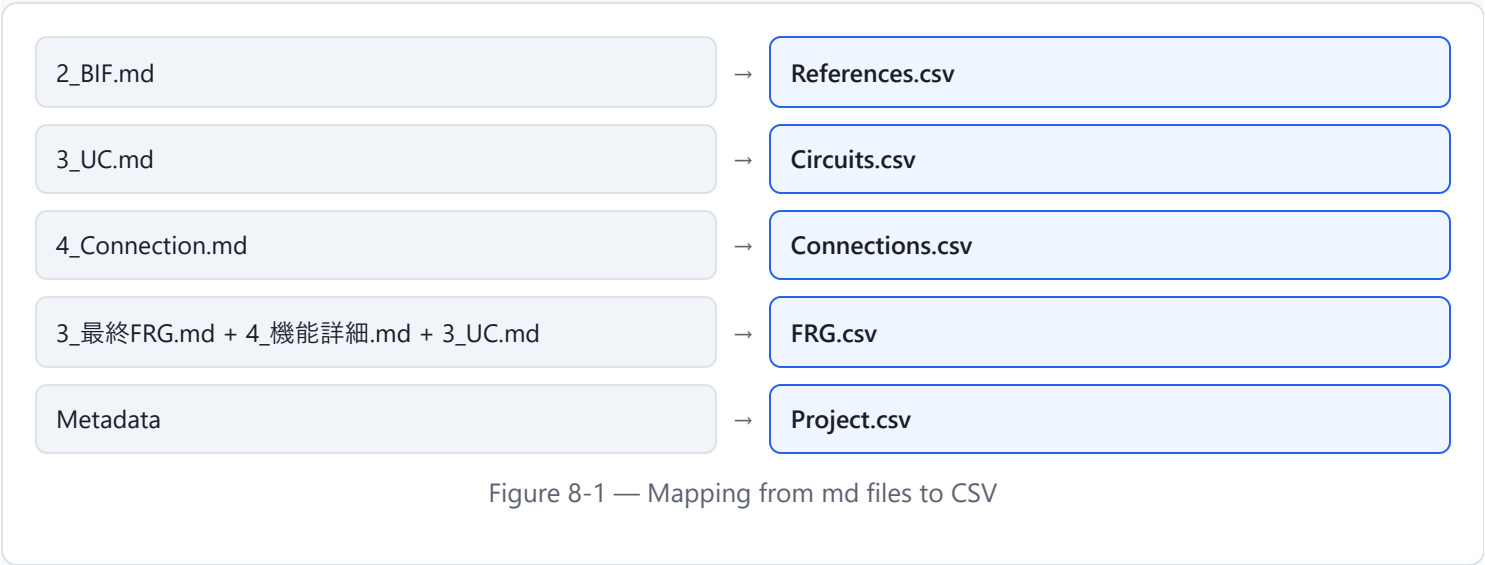
8. Phase 3 — Creating CSV files

See `instruction_3_csv.md`.

Prerequisite files

- HCD: `2_BIF.md`, `3_UC.md`, `4_Connection.md`
- FRG: `3_最終FRG.md`, `4_機能詳細.md`

CSV	Main data sources
Project.csv	Template + metadata
References.csv	2_BIF.md, 4_Connection.md
Circuits.csv	3_UC.md
Connections.csv	4_Connection.md
FRG.csv	FRG + HCD integration



- CSV content must be in **English**
- Remove backticks (``) from markdown
- Add `U.` prefix to UC Node IDs in `FRG.csv`

9. Phase 4 — Generating .bra.xlsx

Run the Python script from the `_latest_version` folder.

```
cd _latest_version
python csv_to_excel.py
```

Interactive input: **Contributor** and **Project ID**

Output: `{ProjectName}/{ProjectName}_CSV/{ProjectID}.bra.xlsx`

Processing performed by the script (summary)

- **Circuits** — insert columns (Sub-Circuits, Uniform=TRUE, etc.); add Contributor / Project ID
- **Connections** — add relation and Notation columns
- **FRG** — merge Capability and Mechanism into `Capability&Mechanism`

10. Using AI agents

Supported tools

CoBRAC work is done with an

AI agent

(Cursor, Claude Code, Codex, or any tool with agent capabilities). Have the agent read each `instruction_*.md` and perform tasks according to those instructions.

Output quality

HCD, FRG, and CSV content generated by AI agents

depends heavily on the model you use

. Higher-capability models tend to produce better literature surveys, consistency checks, and complex functional descriptions. However,

neuroscientific validity and citation accuracy must always be verified by a human

.

Basic workflow

1. Open the CoBRAC repository (`_latest_version`) as the agent working directory
2. Explicitly reference the instruction file for the current Phase
3. Always specify TLF, ROI, Project ID, and other parameters

Prompt examples

Start Phase 1

```
Following instruction_1_HCD.md, start HCD creation with
ROI: hippocampus, TLF: spatial navigation.
Project folders are already created.
```

HCD complete → start FRG

```
HCD is complete. Following instruction_2_FRG.md, start FRG creation
using the TLF and UCs from HCD outputs.
```

FRG complete → create CSV

FRG is complete. Following instruction_3_csv.md,
create all five CSV files.

What humans must verify

- Citation accuracy (DOI, author-year)
- Neuroscientific validity of ROI/TLF
- UC naming rules and GN–UC constraints
- Consistency between Connections and Interfaces

11. Quality checklist

HCD

- ☐ Information path from ROI_Input to ROI_Output is valid
- ☐ Every Connection has a Reference ID
- ☐ Connections match Interfaces
- ☐ UC names use Kebab-case and backticks in markdown
- ☐ Outside-ROI UCs have noROI comments
- ☐ 5_Verification.md completed
- ☐ 8_diagram.md Mermaid diagram created

FRG

- ☐ Only inside-ROI UCs used
- ☐ Each GN decomposed into multiple UCs
- ☐ GN-UC constraints satisfied (max 2 each)
- ☐ TLF Interface consistent with noROI I/O
- ☐ 4_機能詳細.md complete

CSV / xlsx

- ☐ All five CSV files in English
- ☐ No duplicate or missing entries in References.csv
- ☐ Correct Node ID format in FRG.csv (R.* / U.*)
- ☐ .bra.xlsx has five sheets
- ☐ Contributor / Project ID reflected in Circuits and Connections

12. Troubleshooting

Symptom	Cause and fix
CSVフォルダが見つかりません	Check Project ID matches folder name. Path: {ProjectID}/{ProjectID}_CSV/
Project.csv が見つかりません	Phase 3 (CSV) not completed
FRG GN–UC constraint violation	Return to instruction_2 Step 1; refine TLF decomposition
Connection / Interface mismatch	Re-run 5_Verification.md; fix 3_UC.md and 4_Connection.md
ModuleNotFoundError: pandas	Re-run pip install -r requirements.txt
Low AI output quality	Switch to a more capable model; re-reference instructions; review incrementally

Appendix

A. Deliverable file list

File	Phase	Required
1_Thinking.md	HCD	Yes
2_BIF.md	HCD	Yes
3_UC.md	HCD	Yes
4_Connection.md	HCD	Yes
5_Verification.md	HCD	Yes
6_FinalPaper.md	HCD	Yes
7_EasyToUnderstand.md	HCD	Yes
8_diagram.md	HCD	Yes
1_初期分解結果.md	FRG	Yes
2_最適化FRG.md	FRG	Yes
3_最終FRG.md	FRG	Yes
4_機能詳細.md	FRG	Yes
5_FRG作成レポート.md	FRG	Yes
5 × CSV	CSV	Yes
{ProjectID}.bra.xlsx	xlsx	Yes

B. Naming conventions

- **UC** — Kebab-case, no spaces (e.g. flocculus-purkinje)
- **GN** — R. + Kebab-case (e.g. R.error-computation)
- **UC in FRG.csv** — U. + Circuit ID